Mini Project 2: Interactive Web Tool

Start Assignment

- Due Wednesday by 11:59pm
- Points 0
- Submitting a website url
- Available after Oct 6 at 3pm

Build a small tool in JavaScript that remembers something and works out a new answer from it. The user does something, your JavaScript updates the data behind the page, and the page redraws itself.

Submit your repo URL here by **Wednesday 10/21**. The project does not need to be finished by then. It is due **Thursday 10/22**, deployed and working by 3:00 PM, because you will demo it in class that afternoon.

Step 1: create a new public repo

Before any planning or code, create a **new, separate public GitHub repository** for this project. Do not put it inside `oim3690`, `<your-github-username>.github.io` or your Mini Project 1 repo.

Name it in lowercase with hyphens, after what the tool does: `grade-calculator`, `flashcards`. You deploy from this repo with GitHub Pages, so your live URL becomes `https://<your-github-username>.github.io/<repo-name>/`.

Pick something you want to exist

Build a tool that fixes a real annoyance in your week, or one you have wished for. Start from this sentence, and fill in both blanks:

> I wish I had a tool that remembers ___ and tells me ___.

Write three, and pick one after trading them with a classmate in class on Thursday 10/08.

If it is for someone else, name the person who will use it. A real user is the best way to size it: if nobody would open it twice, it is probably too thin.

Adapt these patterns to your own idea.

Too small

- A to-do list that adds, checks and deletes, and nothing else
- A plain countdown or stopwatch
- A one-formula calculator, such as a tip or BMI calculator, with one input and one output

A good size

- A budget tracker that sorts expenses into categories and shows running totals
- A Pomodoro timer that logs finished sessions and shows today's count
- A flashcard app that tracks the cards you missed and moves them to the front
- A grade calculator with weighted categories that tells you the score you still need

Ambitious

- Any of the above, with `localStorage` so the data survives a refresh
- Filtering or sorting, undo, or export to a file
- A small game with rules, a score, and a way to win or lose

A tool of a good size remembers something and works out a new answer from it. One that only reacts to the last click is too small.

Plan the data first

Write `PROPOSAL.md` in your own words before you ask your AI agent for any code:

- **What I am building:** one sentence
- **Who it is for:** a real person, or yourself, and the problem it solves
- **What it remembers:** the data the tool keeps, such as a list of expenses, a score or a deck of cards
- **Where the data lives:** one line, using the table below
- **What each action changes:** for each thing the user can do, what you expect to happen to that data. For example: *Add an expense: a new item joins the list, and that category's total goes up by the amount.*
- **Features:** three to five things the tool will do
- **What I do not know yet:** the JavaScript you expect to have to figure out

Your guesses are not graded on whether they turn out right. When the tool works, add one line to your `README.md` naming where what the agent built differs from what you expected.

Then turn the proposal into `PRD.md` with **From an Idea to a PRD (https://oim3690.github.io/guides/prd-interview/)** , write `DESIGN.md` with **Design Direction (https://oim3690.github.io/guides/design-direction/)** before the agent styles the tool itself, and give the agent its rules in `AGENTS.md`. Every step from a branch to a deployed page is in **Running a Project With an Agent (https://oim3690.github.io/guides/running-a-project/)** .

Where the data lives

Before the agent chooses, answer two questions: who writes the data, and who needs to see it later.

The data	Where it goes
Fixed data you write once: quiz questions, a menu, a set of rates	An array of objects in your JavaScript file

The data	Where it goes
Data the user creates, needed only while the page is open	Variables in your JavaScript, which hold the state
Data the user creates that should survive a refresh	The state, saved to `localStorage`, which keeps it in that one browser
Data several people must see, or that someone edits without touching code	A database or another service, which needs a server and is outside this project

If the agent proposes a database, a `.csv` file or a separate `.json` file, ask it to use an array in your JavaScript file instead. If the idea needs a database to work at all, make it smaller, or save it for your final project.

Design every state

The page changes with its data, and each state needs a look you chose. Describe each one in `PRD.md` before you ask the agent to build it:

- **Empty:** the page when there is nothing to show yet, on a first visit or after the data is cleared. What the page shows, and what it asks the user to do. If your data is written into your JavaScript, such as quiz questions, it is the page before the first answer
- **Invalid input:** whatever your tool should refuse, such as an empty field, a negative amount, text where a number goes, or a move the rules do not allow. Where the message appears, and what it says. If your tool has nothing to refuse, say so in `PRD.md`
- **In use:** the data, and the answer worked out from it
- **After a reset:** what the page returns to

Empty, invalid input and in use are the empty, error and ideal states of the **UI Stack (https://www.scotthurff.com/posts/why-your-user-interface-is-awkward-youre-ignoring-the-ui-stack/)**, a common way to plan the states of a page.

Your JavaScript decides which state the page is in, from the data. The look of each state comes from `DESIGN.md`.

Requirements

- **State** held in JavaScript, such as an array, an object or a running number, that more than one interaction reads or changes
- **Output worked out from that state**, such as a total, a count, a filtered list or a score. Echoing the input back does not count
- At least **three kinds of interaction**: a button, a form, a key press, a dropdown, a checkbox
- The page updates itself in response, with no `alert()` boxes
- Input or moves the tool should refuse are handled: the tool shows what is wrong and keeps working
- A way to reset or start over without reloading the page

- **Every state designed:** the empty page, invalid input, the tool in use and the page after a reset each look the way `PRD.md` describes
- `DESIGN.md` at the top of the repo, and the styles follow it
- Plain HTML, CSS and JavaScript: no React, no Tailwind, no other framework or library, and nothing loaded from a CDN. A project this size is easier to build and to read without one. If your agent adds one, take it out
- No server and no outside API. Everything runs in the browser, from the data in your JavaScript and what the user does. A tool that works on top of another website belongs in Mini Project 4, the browser extension
- Semantic HTML, and readable on a phone
- Deployed on GitHub Pages, and reachable

Who writes the JavaScript

Your agent can write it. The part that holds and changes the state is yours to explain. At the showcase I may point at a line or a block of code in it and ask what happens to the page if you delete it. You answer first, then delete it and see.

Suggested timeline

Two dates are fixed: your repo URL on Canvas by Wednesday 10/21, and the project on Thursday 10/22. Everything else here is pacing.

Suggested pace	Do this
Thursday 10/08	In class, write your wish list, trade it, and pick your idea
Sunday 10/11	`PROPOSAL.md` and `PRD.md` committed, with what the tool remembers, what each action changes, and every state
Tuesday 10/13	`DESIGN.md` committed
Thursday 10/15	One interaction working from start to finish, deployed
Tuesday 10/20	Every interaction built, and your code in class for debugging
Thursday 10/22	Deployed, working, and ready to present

The required question

Every project in this course carries the same question. Answer it in your `README.md`, and again at the showcase:

> Pick one piece of AI output you did not accept as-is. What did it give you, what did you change, and how did you know it needed changing? Point at the commit.
>
> If the agent produced acceptable output on the first attempt: what did you do to verify it?

Answer it about a specific commit in this repo. A general statement about reviewing AI output does not answer it.

Submission

On Canvas, submit the repository URL, and leave the live link for the repo page to show.

Your repo has to contain:

- `PROPOSAL.md`, every section filled in, written by you before the agent wrote any code
- `PRD.md`, written by the agent from your proposal and corrected by you
- `DESIGN.md` at the top of the repo, written before the agent styled the tool itself. The trial pages from Design Direction come before it
- `AGENTS.md` at the top of the repo
- `README.md` with what the tool does, how to use it, the live link, the line on where the agent differed from what you expected, and the required question answered

Then link it from two places:

- **In the repo's About box** (the gear at the top right of the repo page), check **Use your GitHub Pages website** so the live link sits at the top of the repo
- **In your `oim3690` README**, add a line linking to this project

Commit as you go. A single commit holding the finished project shows nothing about how it was built, and your commit history is part of what I read.

How this is graded

The Canvas item collects your repo URL and carries no points of its own. This project is reviewed as part of your checkpoints. **Walkthrough 2** is at the showcase on 10/22. I will ask a few of you a question about your code.

Tips

- Get one interaction working from start to finish before you add the next.
- When the page does not update, `console.log` the state and read what is actually in it.
- Ask the agent for one piece at a time. *Write a function that adds an expense to the list and redraws the totals* gets further than *build me a budget app*.
- When the agent hands you a method you have not seen, look it up on MDN before you keep it.
- Build the empty state first. It is the first thing a new user sees.